

计算机图形学

课程作业 1 光栅化器

[姓名待填写-学号待填写]

一、概述

本作业在教师提供的 C++ 渲染框架中完成单色三角形、规则网格超采样、二维几何变换、质心颜色插值、像素纹理采样和 MipMap 层级采样。六项功能共用三角形覆盖判定和采样缓冲：SVG 中的局部顶点先转换到屏幕坐标，覆盖的子样本再按图元类型写入颜色，最后统一平均到 8-bit RGB 帧缓冲。

主要实现位于 **rasterizer.cpp**、**transforms.cpp** 和 **texture.cpp**。纹理像素采样支持最近邻与双线性，层级采样支持第 0 层、最近层和相邻层线性混合；P 与 L 可独立组合。**L_LINEAR** 是本次任务 6 的必做内容，其中与 P_LINEAR 的组合构成三线性过滤。

实现还修复了起始代码中采样率切换后缓冲大小不匹配、零长度线段处理及最粗 MipMap 层未生成的问题。Windows 工程提供 CMake 编译入口与 VSCode 配置，提交目录同时保留修改源码、可执行程序、报告、机器人 SVG、自备纹理 PNG 和对比图。

实验图由本机 OpenGL 4.6 隐藏窗口自动调用真实 P、L、=、Z、S 键盘回调得到，S 回调读取显示缓冲并保存 PNG。窗口分辨率统一为 800×800，不将自动采集写成人工操作经历。每像素样本数记为 spp：1、4、16 spp 分别对应 1×1、2×2、4×4 网格。

二、绘制三角形

2.1 像素中心与覆盖判定

三条边表示为 $E_i(x,y)=a_ix+b_iy+c_i$ 。按有向面积统一方向，三个边函数都非负时计入覆盖，因而顺、逆时针顶点都正确。只遍历与屏幕相交的包围盒，1 spp 采样像素中心 $(x+0.5,y+0.5)$ 。

边界点包含在内，并允许浮点精度量级的极小容差，避免共享边白缝；退化三角形和非有限坐标直接跳过。

TriangleSetup 预计算系数和面积倒数，子采样行用 $E_i(x+\Delta x,y)=E_i(x,y)+a_i\Delta x$ 增量推进，减少重复算术。三类三角形共用覆盖遍历器，仅着色回调不同。

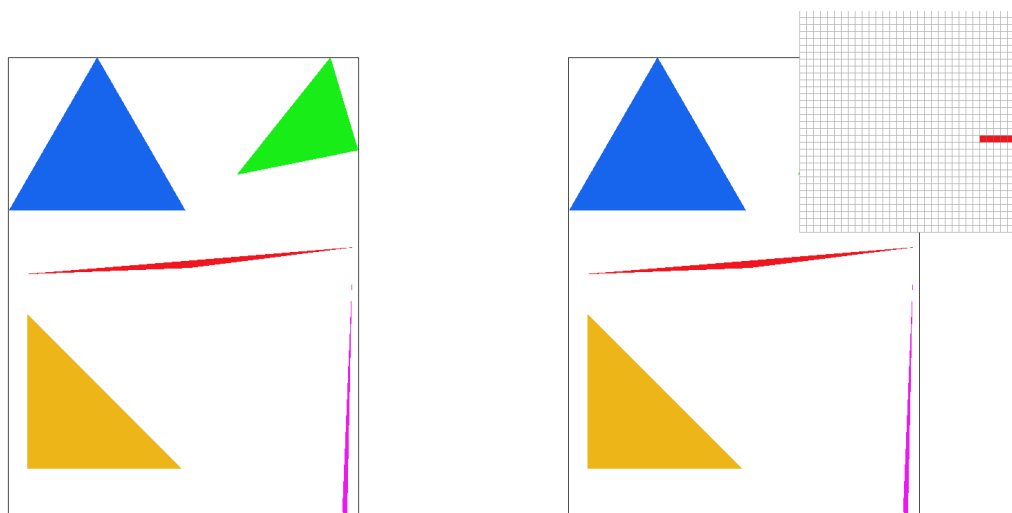


图 2.1 basic/test4.svg，800×800，1 spp，默认视角。右图启用像素检查器，光标固定 (166,378)；整图确认填充，放大区域展示细三角形的阶梯边缘。

2.2 加分实现 方案演进

第一阶段保留直接求值的包围盒基线。每个候选样本都计算 $a_i x + b_i y + c_i$ ，并进行三边判断；它易于核对正确性，但高采样率会重复大量相同系数的计算。

第二阶段使用增量边函数。将系数和面积倒数移到三角形设置阶段，在每个像素的子样本行初始化一次边值，随后以加法推进。这个版本作为优化前的实际实现保留，不能用刻意放慢的基线代替它。

第三阶段增加 **SIMD 并行边函数**。AVX 的一个寄存器容纳四个 double，四个 lane 对应同一行四个相邻像素。每个 lane 仍按原有顺序遍历自己的规则子样本，三个比较结果相与得到覆盖 mask；只对命中的 lane 进行原有颜色或 UV 着色。不是将 double 改成 float，也没有把着色次数改成每像素一次。

AVX 函数独立使用 target 属性，启动后检测 CPU 与操作系统支持。缺少 AVX 时使用两 lane SSE2；其他平台走标量增量路径，行尾不足一组也走标量。禁用 CGL 原有的全局 AVX 自动开关，关闭浮点乘加收缩，使教师机器运行整个程序不以 AVX 为前提。**默认使用 SIMD 模式，B 键可循环切换三个阶段。**

Pineda 的论文给出仿射边函数及并行求值的基础；2016 年的 SIMD 软件遮挡研究提供工程背景。本次没有移植遮挡剔除或深度算法，而是针对当前二维 SVG 的覆盖遍历组织向量运算。分块与扫描线方案经过调研后未采用，便于围绕同一问题比较算法演进。

code/src/rasterizer.cpp

Source excerpt part 1/1 original line numbers

```
45  struct TriangleSetup {
46      double a[3], b[3], c[3];
47      double area, inverse_area, orientation, edge_tolerance;
48      double min_x, max_x, min_y, max_y;
49      bool valid;
50
51      TriangleSetup(float x0, float y0, float x1, float y1,
52                  float x2, float y2) {
53          const double x[3] = {x0, x1, x2};
54          const double y[3] = {y0, y1, y2};
55          valid = true;
56          for (int i = 0; i < 3; ++i) {
57              if (!std::isfinite(x[i]) || !std::isfinite(y[i])) valid = false;
58              const int j = (i + 1) % 3;
59              const int k = (i + 2) % 3;
60              a[i] = y[j] - y[k];
61              b[i] = x[k] - x[j];
62              c[i] = x[j] * y[k] - y[j] * x[k];
63          }
64          area = (x[1] - x[0]) * (y[2] - y[0])
65              - (y[1] - y[0]) * (x[2] - x[0]);
66          valid = valid && std::isfinite(area) && std::abs(area) > 1e-12;
67          inverse_area = valid ? 1.0 / area : 0.0;
68          orientation = area >= 0.0 ? 1.0 : -1.0;
69          edge_tolerance = 16.0 * std::numeric_limits<double>::epsilon()
70                          * std::max(1.0, std::abs(area));
71          min_x = std::min(x[0], std::min(x[1], x[2]));
72          max_x = std::max(x[0], std::max(x[1], x[2]));
73          min_y = std::min(y[0], std::min(y[1], y[2]));
74          max_y = std::max(y[0], std::max(y[1], y[2]));
75      }
76  };
```

图 2.2 三角形设置的实际源码片段。白底代码图从最终源码提取，行号与可复制文本保存在 docs/code-screenshots/。

code/src/rasterizer.cpp

Source excerpt part 1/2 original line numbers

```
204     template <typename ShadeSample>
205     CGL_AVX_FUNCTION void simd4_samples(const TriangleSetup& t, const
    > SampleBounds& box,
206                                     size_t width, unsigned int rate, const
    > ShadeSample& shade) {
207         const __m256d sign = _mm256_set1_pd(t.orientation);
208         const __m256d threshold = _mm256_set1_pd(-t.edge_tolerance);
209         __m256d a[3], c[3], step[3];
210         for (int i = 0; i < 3; ++i) {
211             a[i] = _mm256_set1_pd(t.a[i]); c[i] = _mm256_set1_pd(t.c[i]);
212             step[i] = _mm256_set1_pd(t.a[i] * box.spacing);
213         }
214         for (size_t y = box.yb; y < box.ye; ++y) {
215             size_t x = box.xb;
216             for (; box.xe - x >= 4; x += 4) {
217                 const double offset = .5 * box.spacing;
218                 const __m256d px = _mm256_setr_pd(x + offset, x + 1 + offset,
219                                                    x + 2 + offset, x + 3 + offset);
220                 for (unsigned int sy = 0; sy < box.side; ++sy) {
221                     const double py = y + (sy + .5) * box.spacing;
222                     __m256d edge[3];
223                     for (int i = 0; i < 3; ++i)
224                         edge[i] = _mm256_add_pd(_mm256_add_pd(_mm256_mul_pd(a[i], px),
225                                                                    _mm256_set1_pd(t.b[i] * py)), c[i]);
226                     for (unsigned int sx = 0; sx < box.side; ++sx) {
227                         __m256d inside = _mm256_cmp_pd(_mm256_mul_pd(sign, edge[0]),
    > threshold, _CMP_GE_OQ);
```

图 2.3 四 lane AVX 覆盖测试的实际源码。完整函数与后续片段见同目录源码图及 code/src/rasterizer.cpp。

code/src/rasterizer.cpp

Source excerpt part 2/2 original line numbers

```
228         inside = _mm256_and_pd(inside, _mm256_cmp_pd(_mm256_mul_pd(sign,
> edge[1]), threshold, _CMP_GE_OQ));
229         inside = _mm256_and_pd(inside, _mm256_cmp_pd(_mm256_mul_pd(sign,
> edge[2]), threshold, _CMP_GE_OQ));
230         const int mask = _mm256_movemask_pd(inside);
231         if (mask) {
232             double values[3][4];
233             for (int i = 0; i < 3; ++i) _mm256_storeu_pd(values[i],
> edge[i]);
234             for (int lane = 0; lane < 4; ++lane) if (mask & (1 << lane)) {
235                 const double e[3] = {values[0][lane], values[1][lane],
> values[2][lane]};
236                 const size_t index = (y * width + x + lane) * rate + sy *
> box.side + sx;
237                 shade_edges(t, index, e, shade);
238             }
239         }
240         for (int i = 0; i < 3; ++i) edge[i] = _mm256_add_pd(edge[i],
> step[i]);
241     }
242 }
243 }
244     for (; x < box.xe; ++x) scalar_pixel<false>(t, box, width, rate, x, y,
> shade);
245 }
246 }
```

图 2.3 续图：将覆盖 mask 中命中的 lane 写入逐子样本缓冲，并处理不足四个像素的行尾。

2.3 正确性与性能评估

环境为 Windows x64、AMD Ryzen AI 7 H 350、GCC 15.1.0，RelWithDebInfo 和 -ffp-contract=off；本机选择 AVX 四路 double。固定种子 20261002，四组输入分别含 10 个大三角形、512 个亚像素至小尺寸三角形、12 个细长三角形、12 个越界裁剪三角形，并交替顶点方向。分辨率 512²、1024²，计时采样率 1、4、16 spp；预热 2 次，每种模式重复 11 次，轮换执行顺序，报告中位数。计时包括真实 rasterize 调用的设置、覆盖测试、单色着色与采样缓冲写入，**不包括清空、resolve、SVG 解析、窗口、CSV 或 PNG 导出**，不能将其称为程序整帧耗时。

独立 long-double 端点叉积判定在固定 24×24 校验窗口中核对 72 个边界、裁剪、退化和随机三角形，包含 1/4/9/16 spp 与三个内核，共 864 种配置；两个基准运行各核对一次，合计 1728 次，错误样本为 0。实际 512²/1024² 窗口中，单色、质心颜色及纹理三条管线合计 384 组对照，覆盖与最终 RGB 字节差异为 0，SIMD 相对增量版本的内部 float 颜色样本也完全同值。强制增量模式实际验证标量路径；当前 AVX 机器未模拟无 AVX 硬件，SSE2 回退的编译成功不等同于跨机器执行验证。

表 2.1 任务一主结果，1 spp。相对增量 = 增量中位耗时 / SIMD 中位耗时，超过 1 表示加速。

场景	分辨率	直接/ms	增量/ms	SIMD/ms	相对增量
大面积	512 ²	5.7067	6.3670	3.7079	1.72×
小三角形	512 ²	0.0420	0.0406	0.0498	0.82×
细长	512 ²	9.7232	10.7194	5.0770	2.11×
裁剪	512 ²	9.7943	10.5884	6.3410	1.67×
大面积	1024 ²	29.3161	31.5610	19.3601	1.63×
小三角形	1024 ²	0.0539	0.0621	0.0684	0.91×
细长	1024 ²	41.4859	48.0910	25.1230	1.91×
裁剪	1024 ²	54.4473	59.6680	37.3478	1.60×

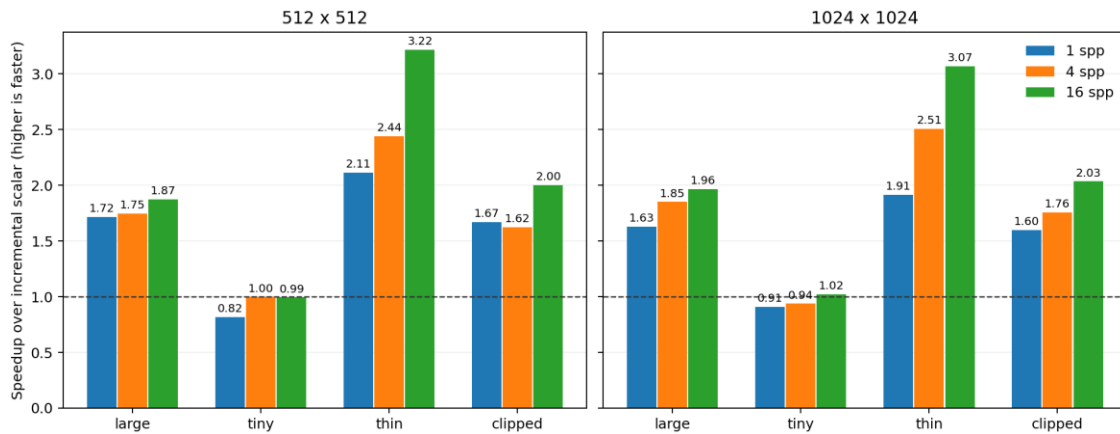


图 2.4 两分辨率、1/4/16 spp 的 SIMD 相对增量加速比；虚线为 1。数据来自实际 chrono 测量，柱顶给出数值，完整 p10/p90、极值与逐次耗时保存在 verification/performance/。

大三角形、细长及裁剪场景提供足够多的相邻位置，向量算术与覆盖 mask 减少标量比较开销。**小三角形可出现退化：**较短的行很快落入标量尾部，向量设置、调度与条件散写无法摊薄；该输入的总时间也很短，更易受计时噪声影响。1 spp 时没有像素内部的增量推进收益，增量阶段本身未必比直接求值快，因此必须保留两个实际基准，不能把每一步演进都写成必然加速。

此 SIMD 实现保持逐样本着色，改善的是覆盖计算，未解决 AoS 颜色散写或纹理读取带宽。收益依赖几何、采样率、CPU 与编译器，不沿用文献的加速数字。原始几何、校验、计时和源码哈希全部保留，可通过 `scripts/benchmark_simd.py --run` 重现。

三、基于超采样的反走样算法

3.1 规则网格与缓冲布局

令 $S = \text{sample_rate}$ 、 $n = \sqrt{S}$ 。每个像素的 $n \times n$ 子样本均匀分布在小格中心，第 (sx, sy) 个样本的位置为 $x + (sx + 0.5)/n$ 、 $y + (sy + 0.5)/n$ 。规则网格是本次作业要求的超采样方法；不以非规则样本或仅覆盖率统计替代它。

`sample_buffer` 将每个像素的 S 个 `Color` 连续存储，像素按行优先排列，子样本也按行优先排列。因此索引为 $(y \times \text{width} + x) \times S + sy \times n + sx$ 。构造、`set_sample_rate` 和 `set_framebuffer_target` 都按 $\text{width} \times \text{height} \times S$ 分配；采样率必须是正的平方数，尺寸计算检查溢出。

每帧开始将帧缓冲和采样缓冲清为白色。三角形仅改写命中的子样本；点和线通过 `fill_pixel` 把对应像素的所有子样本写为同色，使原有轮廓在提高采样率后仍可见。所有图元完成后，`resolve_to_framebuffer` 对同一像素的 S 个颜色求均值，再夹到 $[0, 1]$ 并四舍五入为 0–255。

与直接写最终像素相比，这一管线保留了图元之间的逐子样本覆盖关系。边缘像素的最终颜色由被覆盖样本比例决定。例如白底上的红三角形覆盖 16 个样本中的 8 个，平均得到红色与白色各半的颜色，而不是整像素突然在红白之间跳变。

缓冲空间为 $\text{width} \times \text{height} \times S \times \text{sizeof}(\text{Color})$ ，相对于每像素样本数 S 线性增长。若只存一个累计颜色，后续遮挡图元便无法知道哪些子样本已被覆盖；分块或瓦片缓冲可以节约峰值内存，但要重新组织绘制顺序与 `resolve`。

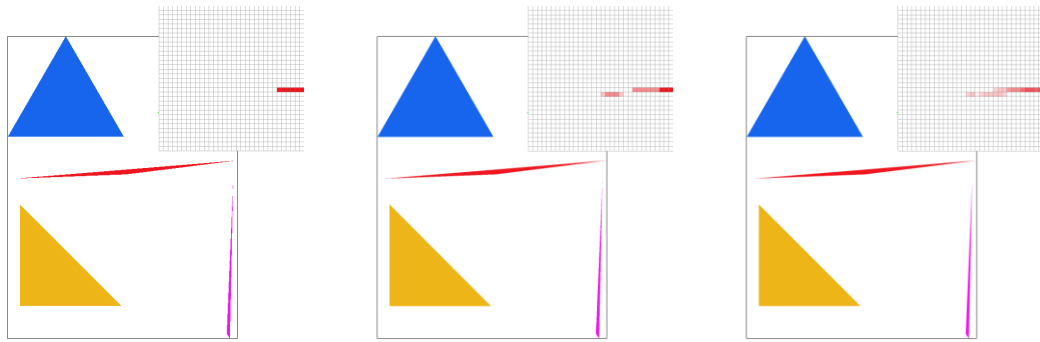


图 3.1 basic/test4.svg, 800×800, 默认视角, 1、4、16 spp 并列比较。三图都启用像素检查器, 光标固定 (166,378), 分别使用 1×1、2×2、4×4 子样本网格。

3.2 观察与代价

1 spp 只有一个“命中或未命中”的判断, 斜边形成明显台阶, 细尖角也容易断续。4 spp 提供五档覆盖比例, 边缘开始出现过渡; 16 spp 提供十七档覆盖比例, 细三角形的变化更连续。这里比较的是固定几何与固定视角下的采样密度, 并非把不同缩放造成的视觉变化归于抗锯齿。

更高采样率同时提高覆盖测试、颜色插值、纹理过滤和 **resolve** 的工作量。规则网格仍是有限采样, 对于比子格更细的高频结构可能留下走样; 超采样改善误差, 并不保证所有场景都完全没有锯齿。

code/src/rasterizer.cpp

Source excerpt part 1/2 original line numbers

```
296 void RasterizerImp::fill_pixel(size_t x, size_t y, Color c) {
297     // Points and lines cover the entire pixel, so broadcast to every
    > sample.
298     if (x >= width || y >= height) return;
299     const size_t base = (y * width + x) * sample_rate;
300     std::fill(sample_buffer.begin() + base,
301               sample_buffer.begin() + base + sample_rate, c);
302 }

397 void RasterizerImp::set_sample_rate(unsigned int rate) {
398     rate = validate_sample_rate(rate);
399     // Allocate first to keep the previous state consistent on failure.
400     std::vector<Color> replacement(sample_count(width, height, rate),
    > Color::White);
401     sample_buffer.swap(replacement);
402     sample_rate = rate;
403 }

420 void RasterizerImp::set_framebuffer_target(unsigned char* rgb_framebuffer,
421     size_t width, size_t height)
422 {
423     std::vector<Color> replacement(sample_count(width, height, sample_rate),
    > Color::White);
```

图 3.2 采样缓冲平均与 8-bit 输出的实际源码。16 spp 对应 4×4 网格，不能与 16×16 混用。

code/src/rasterizer.cpp

Source excerpt part 2/2 original line numbers

```
424     sample_buffer.swap(replacement);
425     this->width = width;
426     this->height = height;
427     this->rgb_framebuffer_target = rgb_framebuffer;
428 }

444 void RasterizerImp::resolve_to_framebuffer() {
445     if (!rgb_framebuffer_target) return;
446     const float weight = 1.0f / sample_rate;
447     for (size_t pixel = 0; pixel < width * height; ++pixel) {
448         Color col;
449         const size_t base = pixel * sample_rate;
450         for (unsigned int sample = 0; sample < sample_rate; ++sample) {
451             col += sample_buffer[base + sample];
452         }
453         col *= weight;
454         rgb_framebuffer_target[3 * pixel] = encode_channel(col.r);
455         rgb_framebuffer_target[3 * pixel + 1] = encode_channel(col.g);
456         rgb_framebuffer_target[3 * pixel + 2] = encode_channel(col.b);
457     }
458
459 }
```

图 3.2 续图：完成样本平均、夹取与舍入。采样率改变和窗口 `resize` 使用相同缓冲布局。

3.3 扩展实验 解析覆盖率

除必做规则 SSAA 外，增加可通过 **A 键切换的解析覆盖率模式**。对单位像素方格 P 与三角形 T 求交，多边形面积 $A=\text{area}(P\cap T)$ 就是 box filter 下的几何覆盖率。使用半平面连续裁剪求交集，又积累计面积及一阶矩求质心；几何运算使用 double，不依赖规则子样本是否恰好击中细尖角。

白底上单色图形的像素颜色为 $A\times C+(1-A)\times\text{白色}$ 。但若直接对每个三角形独立执行这条混色公式，两块拼成矩形的三角形会在内部对角线上出现接缝，重叠图元也会将被遮挡的颜色混入结果。**因此，本实现先求可见区域，再对颜色积分。**

按图元绘制顺序从后向前处理，初始“尚未覆盖区域”为整个像素方格。当前三角形与它求交，贡献可见子区域的颜色；随后将三角形从尚未覆盖区域中扣除。剩余区域继续与前面的图元求交，最后剩余面积显示白色背景。凸区域相减沿三条边分解为互不重叠的子多边形，允许同一像素含多个分离区域。这保留了不透明图元的覆盖关系，也正确处理 SVG 多边形内部的三角剖分共享边。

对于常量色，面积乘颜色就是精确积分。对于仿射 RGB 插值，区域均值等于在该可见区域质心处的颜色，因此面积与质心也足以积分。**纹理在可见子区域质心处取样，其几何覆盖精确，但纹理积分是近似；不能把它写为任意纹理都精确。**点与线保留原作业允许的整像素处理，细线实验使用狭长填充三角形，不声称一般 stroke 已实现解析抗锯齿。透明度和一般 SVG 曲线路径也不在本次扩展范围。

code/src/analytic_rasterizer.cpp

Source excerpt part 1/1 original line numbers

```
16 ClipPolygon clip(const ClipPolygon& polygon, const Vector2D& a, const
  > Vector2D& b,
17     double orientation, bool inside) {
18     ClipPolygon result;
19     if (polygon.empty()) return result;
20     result.reserve(polygon.size()+1);
21     Vector2D previous = polygon.back();
22     double ep = orientation * cross(a,b,previous);
23     bool previous_in = inside ? ep >= 0 : ep <= 0;
24     for (const Vector2D& current : polygon) {
25         const double ec = orientation * cross(a,b,current);
26         const bool current_in = inside ? ec >= 0 : ec <= 0;
27         if (current_in != previous_in) {
28             const double t = ep / (ep-ec);
29             result.emplace_back(previous.x + t*(current.x-previous.x),
30                               previous.y + t*(current.y-previous.y));
31         }
32         if (current_in) result.push_back(current);
33         previous=current; ep=ec; previous_in=current_in;
34     }
35     return result;
36 }
```

图 3.3 半平面裁剪源码：与像素方格求交，通过面积计算连续覆盖率。

code/src/analytic_rasterizer.cpp

Source excerpt part 1/2 original line numbers

```
67 void subtract(ClipPolygon region, const Vector2D v[3], double orientation,
68               std::vector<ClipPolygon>& outside) {
69     for (int e=0; e<3 && !region.empty(); ++e) {
70         ClipPolygon piece=clip(region,v[e],v[(e+1)%3],orientation,false);
71         Vector2D ignored;
72         if (measure(piece,ignored)>1e-15) outside.push_back(std::move(piece));
73         region=clip(region,v[e],v[(e+1)%3],orientation,true);
74     }
75 }

194 void AnalyticRasterizer::resolve_to_framebuffer() {
195     if(!framebuffer)return;
196     for(size_t y=0;y<height;++y)for(size_t x=0;x<width;++x) {
197         const auto& bin=tile_bins[(y/tile_side)*tiles_x+x/tile_side];
198         if(bin.empty())continue;
199         std::vector<ClipPolygon> uncovered{pixel_square(x,y)};
200         double rgb[3]={0,0,0};
201         // Traverse from last drawn to first. Earlier layers only contribute
202         > where
203         // later opaque geometry is absent. This avoids seams along
204         > tessellation.
205         for(auto it=bin.rbegin();it!=bin.rend()&&!uncovered.empty();++it) {
206             const Primitive& p=primitives[*it];
```

图 3.4 可见区域积分源码：按绘制顺序扣除遮挡，累计面积与颜色。

code/src/analytic_rasterizer.cpp

Source excerpt part 2/2 original line numbers

```
205     if(int(x)<p.x0||int(x)>=p.x1||int(y)<p.y0||int(y)>=p.y1)continue;
206     const double orientation=p.area>0?1:-1;
207     std::vector<ClipPolygon> next;
208     for(const ClipPolygon& region:uncovered) {
209         ClipPolygon visible=intersection(region,p.v,orientation);
210         Vector2D centroid;const double area=measure(visible,centroid);
211         if(area<=1e-15){next.push_back(region);continue;}
212         const Color c=shade(p,centroid);
213         rgb[0]+=area*c.r;rgb[1]+=area*c.g;rgb[2]+=area*c.b;
214         subtract(region,p.v,orientation,next);
215     }
216     uncovered.swap(next);
217 }
218 // The remaining exact area shows the white background.
219 for(const ClipPolygon& region:uncovered) {
220     Vector2D centroid;const double area=measure(region,centroid);
221     for(double& v:rgb)v+=area;
222 }
223 const size_t i=3*(y*width+x);
224 framebuffer[i]=byte(rgb[0]);framebuffer[i+1]=byte(rgb[1]);
225 > framebuffer[i+2]=byte(rgb[2]);
226 }
```

图 3.4 续图：扣除已经可见的区域，将剩余白底面积计入最终颜色。

3.4 画质与成本评估

20 项解析检查通过，含手算半像素、顺逆时针、零面积、0.01 像素面积的细三角形、大片坐标稳定性、共享边无缝、不透明重叠顺序、仿射 RGB 质心积分、边界裁剪、末像素线段、清空及 **resize** 安全性。对于单位直角三角形，面积 0.5，黑白平均得到 128；对于 RGB 三顶点，覆盖区域质心处三色各 1/3，乘面积后加白底得到每通道 170。

数值对比使用 96×96 帧缓冲、五个固定场景和四种模式（1/4/16 spp SSAA、解析覆盖率）。参考程序独立在每像素 64×64 中心网格上直接查找最后覆盖的三角形并着色，**4096 spp 参考仍是有限采样，不能称为数学真值**。误差用相对参考的全图 RGB 平均绝对误差 MAE，单位为 0–255 通道值。解析几何的舍入及参考采样误差使 MAE 不必恰为零。

表 3.1 画质与耗时；MAE 越小越接近独立数值参考。耗时含 **clear**、提交图元及 **resolve**，2 次预热、11 次重复、中位数；与表 2.1 的覆盖计时范围不同，不能直接横向比较。

场景	SSAA 1 MAE	SSAA 16 MAE	解析 MAE	SSAA 16/ms	解析/ms
斜边	1.7180	0.1998	0.0084	0.205	2.002
细三角形	0.3288	0.0973	0.0037	0.155	0.813
共享边	1.9258	0.2865	0.0083	0.242	2.493
重叠	1.8866	0.2450	0.0094	0.271	2.772
插值及裁剪	1.2036	0.2689	0.1381	0.412	2.713

解析覆盖率能保留未击中任何规则样本的细几何，并在共享边与重叠处按可见区域积分。代价是像素内多边形裁剪、分裂、面积矩和临时分配；本实现重视可核对的正确性，不将解析模式宣称为实时速度优势。复杂重叠会增加碎片数量，后续可加入全覆盖快速路径和固定容量临时多边形存储。

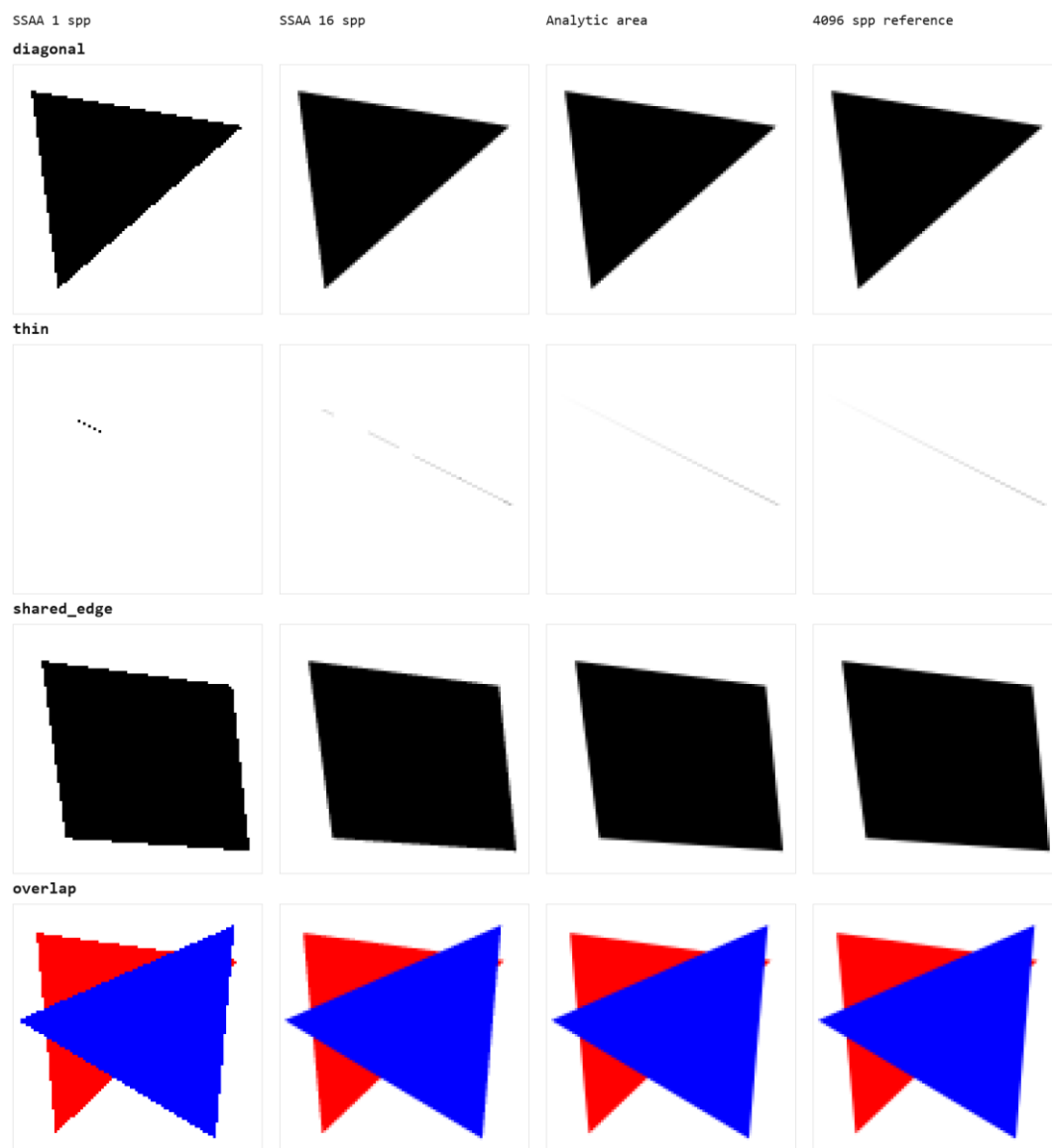


图 3.5 四类几何场景，从左到右为 SSAA 1、SSAA 16、解析面积和独立 4096 spp 参考。
 图由实际 96×96 framebuffer PNG 进行 4 倍最近邻放大，方便检查像素；这是数值实验图，不是 GUI 截图。

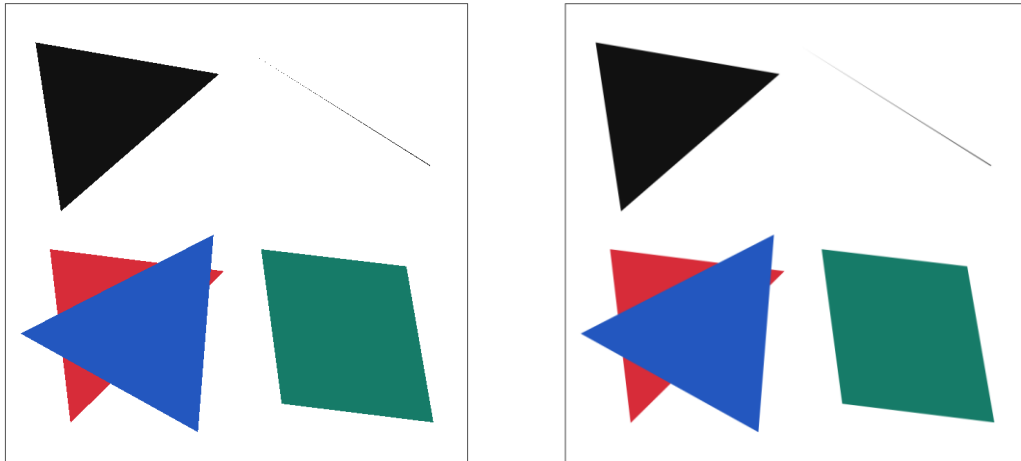


图 3.6 自建 docs/analytic_demo.svg，800×800、相同默认视角，左为 1 spp SSAA，右为 A 键解析模式。通过隐藏 GLFW 窗口的真实 A/S 回调取得无损 PNG，参数与 GL 检查记录在 verification/aa/gui-captures.json。

表 3.2 96×96 单独渲染器的持久存储比较。

模式	持久几何或样本存储	计时及精度范围
SSAA 1	110592 B（108 KiB）	1 个中心样本/像素
SSAA 4	442368 B（432 KiB）	2×2 规则样本/像素
SSAA 16	1769472 B（1728 KiB）	4×4 规则样本/像素
解析模式	1184–1792 B	本五场景的几何与 16×16 瓦片索引容量

四种模式均另需相同的 RGB framebuffer 27648 B。表中未包含临时裁剪多边形、vector 分配器开销或进程峰值；GUI 为便于切换同时持有 SSAA 和解析对象，因此 GUI 进程内存不等于这张表的单独渲染器存储。实验原始计时、RMSE、最大误差、全部模式输出 PNG 与哈希留存在 verification/aa/。

四、几何变换

4.1 齐次矩阵与关节层级

顶点使用齐次列向量 $(x,y,1)$ 。平移量位于矩阵第三列，缩放量位于主对角线；旋转角度乘 $\pi/180$ 后以 $\cos\theta$ 、 $\sin\theta$ 构造矩阵。 $M=T\times R\times S$ 依次缩放、旋转、平移。

SVG 父、子分组变换相乘，让部件绕自己的关节运动。绕 (cx,cy) 旋转为 $T(cx,cy)\times R\times T(-cx,-cy)$ 。原 robot.svg 核对基本矩阵，docs/my_robot.svg 保存可编辑的新动作。

新小人边跑边挥手：身体倾斜 -10° ，右上臂/前臂局部旋转 $-65^\circ/-35^\circ$ ；大腿为 $32^\circ/-30^\circ$ ，小腿再旋转 $-65^\circ/30^\circ$ 。头部再转 10° ，抵消身体倾斜。衣服蓝色，上臂青绿，头和手橙黄。

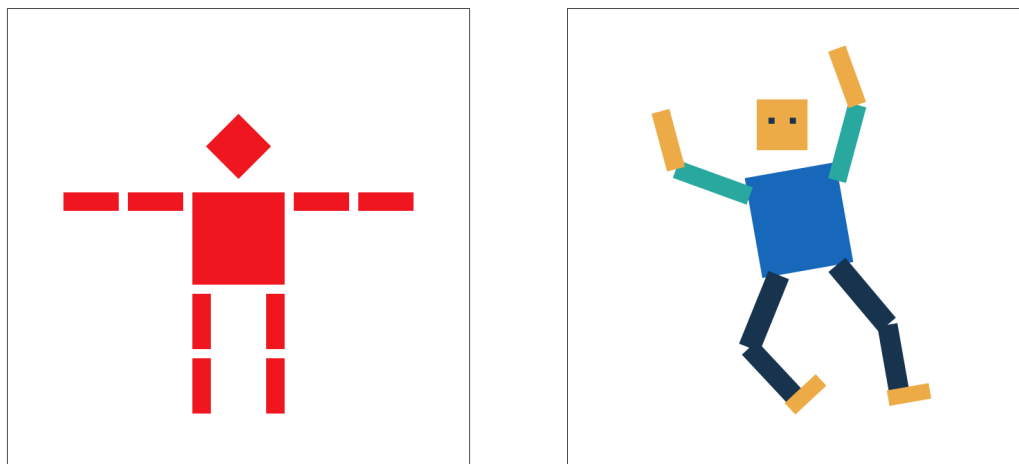


图 4.1 原 robot.svg 与 docs/my_robot.svg，800×800，各自默认视角，16 spp。两图具有相同输出分辨率和采样数，可对照部件关系与姿态。

code/src/transforms.cpp

Source excerpt part 1/1 original line numbers

```
16 Matrix3x3 translate(float dx, float dy) {
17     return Matrix3x3(1, 0, dx,
18                     0, 1, dy,
19                     0, 0, 1);
20 }

22 Matrix3x3 scale(float sx, float sy) {
23     return Matrix3x3(sx, 0, 0,
24                     0, sy, 0,
25                     0, 0, 1);
26 }

29 Matrix3x3 rotate(float deg) {
30     const double radians = deg * std::acos(-1.0) / 180.0;
31     const double c = std::cos(radians);
32     const double s = std::sin(radians);
33     return Matrix3x3(c, -s, 0,
34                     s,  c, 0,
35                     0,  0, 1);
36 }
```

图 4.2 平移、缩放和角度制旋转矩阵的实际源码。

4.2 GUI 加分 视口中心旋转

新增 Q/E 键，分别逆时针、顺时针旋转 15° 。它改变当前 SVG 的观察视口，不修改机器人关节或源文件。每个加载的 SVG 独立保存中心 (cx,cy)、跨度 span 与角度 θ ，数字键切换时保留各自状态；Space 恢复当前 SVG 的初始中心、跨度及零角度。

原 M_{base} 将观察中心映射到 NDC 的 (0.5,0.5)。新的 SVG→NDC 矩阵为 $T(0.5,0.5) \times R(\theta) \times T(-0.5,-0.5) \times M_{\text{base}}$ 。NDC→screen 仍按 $\min(\text{width}, \text{height})$ 等比缩放并居中，因此完整栈为 $M_{\text{screen}} \times T(0.5) \times R(\theta) \times T(-0.5) \times M_{\text{base}} \times M_{\text{element}}$ 。屏幕 y 轴向下，正角度在屏幕上表现为顺时针。

旋转后不能再从矩阵平移项直接反推观察中心，所以使用显式 ViewState。拖动时，将屏幕位移经 $R(-\theta)$ 和 $2\text{span}/\min(\text{width}, \text{height})$ 换算为 SVG 位移；缩放只改变 span，并保留中心与角度。这样旋转后拖动方向仍跟随鼠标，窗口改变大小也不会拉伸图形。

```
code/src/drawrend.cpp
Source excerpt part 1/1 original line numbers

496 void DrawRender::set_view(float x, float y, float span) {
497     if (!std::isfinite(x) || !std::isfinite(y) ||
498         !std::isfinite(span) || span <= 0) return;
499     ViewState& view = view_states[current_svg];
500     view.x = x; view.y = y; view.span = span;
501     // The original SVG->NDC transform maps the view center to (0.5,0.5).
502     // Rotate around that point before the unchanged, isotropic NDC->screen
    > map.
503     const Matrix3x3 base(1, 0, -x + span, 0, 1, -y + span, 0, 0, 2 * span);
504     svg_to_ndc[current_svg] = translate(.5f, .5f) * rotate(view.angle)
505                             * translate(-.5f, -.5f) * base;
506 }

518 void DrawRender::rotate_view(float degrees) {
519     if (!std::isfinite(degrees)) return;
520     ViewState& view = view_states[current_svg];
521     view.angle = std::remainder(view.angle + degrees, 360.0f);
522     set_view(view.x, view.y, view.span);
523 }
```

图 4.3 视口矩阵与显式中心、跨度、角度状态的实际源码。



图 4.4 docs/my_robot.svg, 800×800、16 spp。左为初始视图，中为 E 两次后的顺时针 30° 视图，右为 Space 复位结果；原始 SVG 未改动。

13 项自动检查全部通过，覆盖中心位置与旋转半径不变、Q 撤销 E、24 次旋转一圈、旋转后的拖动与缩放、非正方形窗口、每 SVG 状态、复位及 A/B 模式切换。所有 OpenGL 检查均无错误。**复位图与初始图逐像素完全一致**，原 19 张报告 PNG 哈希也保持不变。日志、矩阵条件与截图参数留存在 verification/viewport-tests.json 与 viewport-tests.log；这是真实隐藏 GLFW 窗口中的程序回调验证，不是人工按键经历。

五、质心坐标

5.1 面积权重与颜色插值

对顶点 V_0 、 V_1 、 V_2 ，点 P 的质心坐标满足 $P=\alpha V_0+\beta V_1+\gamma V_2$ ，且 $\alpha+\beta+\gamma=1$ 。三角形内部的三个权重非负；在某个顶点处，对应权重为 1，在它的对边处为 0。它们也可理解为对应子三角形的有向面积与总面积之比。

覆盖遍历器将三个对边函数除以有向面积，得到 α 、 β ，并以 $\gamma=1-\alpha-\beta$ 计算第三个权重。颜色插值为 $C(P)=\alpha C_0+\beta C_1+\gamma C_2$ 。由于颜色按样本计算， 4×4 超采样时每个样本都得到自己的插值颜色，再在像素内平均。

如果三个顶点分别为红、绿、蓝，接近哪个顶点就得到更多对应分量，重心处三个分量相等，形成平滑渐变。`test7.svg` 将多个颜色三角形组成色轮，用于检验内部插值与共享边的连续性。

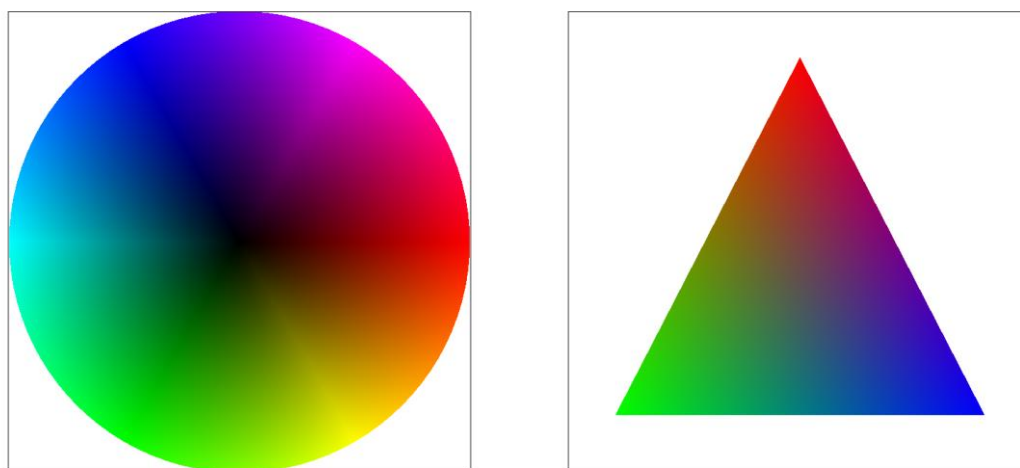


图 5.1 basic/test7.svg 与 docs/barycentric.svg， 800×800 ，默认视角，1 spp。红绿蓝顶点的颜色权重在线性插值中连续变化。

code/src/rasterizer.cpp

Source excerpt part 1/1 original line numbers

```
357 void RasterizerImp::rasterize_interpolated_color_triangle(float x0, float
    > y0, Color c0,
358     float x1, float y1, Color c1,
359     float x2, float y2, Color c2)
360 {
361     const TriangleSetup triangle(x0, y0, x1, y1, x2, y2);
362     for_each_triangle_sample(triangle, width, height, sample_rate,
    > rasterization_method,
363     [&](size_t sample, double w0, double w1, double w2) {
364         sample_buffer[sample] = c0 * static_cast<float>(w0)
365                                 + c1 * static_cast<float>(w1)
366                                 + c2 * static_cast<float>(w2);
367     });
368 }
```

图 5.2 顶点颜色按质心权重插值的实际源码。

六、基于像素采样的纹理映射

6.1 屏幕样本到纹素

屏幕样本以质心权重求 $uv(P)=\alpha uv_0+\beta uv_1+\gamma uv_2$ ，再由 `Texture::sample` 从离散纹素重建颜色。

UV 两端对应首末纹素中心， $x=u(W-1)$ 、 $y=v(H-1)$ ，越界值夹到边缘。最近邻将 x 、 y 四舍五入，读取一个纹素。双线性读取四邻纹素，用小数 tx 、 ty 先混合上下两行，再按 ty 混合两行；边界邻点也夹取。

例如 $top=(1-tx)C_{00}+txC_{10}$ ，最终 $C=(1-ty)top+ty\times bottom$ 。MipLevel 以引用访问，避免复制图像。

本组选择 `texmap/test4.svg` 鹦鹉，L_ZERO、缩放 0.4，检查器置于眼部边缘 (348,230)。缺失的 PNG 从原框架固定提交恢复，地址与 SHA256 见 `dependency-assets.json`。



图 6.1 `texmap/test4.svg`，800×800，缩放系数 0.4，检查器 (348,230)，L_ZERO、1 spp；
左 P_NEAREST，右 P_LINEAR。



图 6.2 同一场景、视角、检查器与 800×800 分辨率，L_ZERO、16 spp：左 P_NEAREST，右 P_LINEAR。

```
code/src/texture.cpp
Source excerpt  part 1/2  original line numbers
72  Color Texture::sample_nearest(Vector2D uv, int level) {
73      if (level < 0 || static_cast<size_t>(level) >= mipmap.size() ||
74          !std::isfinite(uv.x) || !std::isfinite(uv.y)) return Color(1, 0, 1);
75      auto& mip = mipmap[level];
76      if (!valid_mip(mip)) return Color(1, 0, 1);
77
78      // In this framework UV endpoints map to the first and last texel
79      > centers.
80      const double x = clamp_uv(uv.x) * (mip.width - 1);
81      const double y = clamp_uv(uv.y) * (mip.height - 1);
82      const int tx = static_cast<int>(std::floor(x + 0.5));
83      const int ty = static_cast<int>(std::floor(y + 0.5));
84      return mip.get_texel(tx, ty);
85  }

86  Color Texture::sample_bilinear(Vector2D uv, int level) {
87      if (level < 0 || static_cast<size_t>(level) >= mipmap.size() ||
88          !std::isfinite(uv.x) || !std::isfinite(uv.y)) return Color(1, 0, 1);
```

图 6.3 最近邻与双线性重建的实际源码。续图和可复制文本与截图一同保存。

code/src/texture.cpp

Source excerpt part 2/2 original line numbers

```
89     auto& mip = mipmap[level];
90     if (!valid_mip(mip)) return Color(1, 0, 1);
91
92     const double x = clamp_uv(uv.x) * (mip.width - 1);
93     const double y = clamp_uv(uv.y) * (mip.height - 1);
94     const int x0 = static_cast<int>(std::floor(x));
95     const int y0 = static_cast<int>(std::floor(y));
96     const int x1 = std::min(x0 + 1, static_cast<int>(mip.width) - 1);
97     const int y1 = std::min(y0 + 1, static_cast<int>(mip.height) - 1);
98     const float tx = static_cast<float>(x - x0);
99     const float ty = static_cast<float>(y - y0);
100    const Color top = mip.get_texel(x0, y0) * (1.0f - tx) +
    > mip.get_texel(x1, y0) * tx;
101    const Color bottom = mip.get_texel(x0, y1) * (1.0f - tx) +
    > mip.get_texel(x1, y1) * tx;
102    return top * (1.0f - ty) + bottom * ty;
103 }
```

图 6.3 续图：双线性对四邻纹素分两次混合，并处理纹理边界。

6.2 同场景对比

在纹理被放大时，一枚纹素占据多个屏幕像素，最近邻的阶梯和块感最明显；双线性在纹素之间形成连续过渡，因而更适合本组放大区域。代价是突变边界和细小笔画会变柔和。**更平滑不等于能够恢复原图中没有的细节。**

将 1 spp 提高到 16 spp，可以平均同一屏幕像素中的多次 UV 取样，改善几何边缘和纹理边界。但如果十六个样本仍落在同一个纹素邻域，多次取样也不会增加源图细节。双线性控制纹理域的重建，超采样控制屏幕像素内的积分，两者处理的范围不同，可以组合使用。

纹理被大幅缩小时，一个屏幕像素覆盖许多纹素，最近邻只读取一个、双线性只混合四个，都可能遗漏高频信息并产生摩尔纹。该情形需要下一节的层级采样先使用经过低通过滤的纹理层。

七、MipMap 纹理映射

7.1 足迹与层级选择

MipMap 将原图预先过滤为逐级降低分辨率的图像。层级采样根据屏幕像素在纹理上的足迹选择分辨率，使缩小后的像素取到更接近区域平均的颜色。它与某一层内的最近邻或双线性采样独立，因此可组合成六种模式。

在每个样本处记录 $uv(P)$ 、 $uv(P+(1,0))$ 、 $uv(P+(0,1))$ 。本实现利用质心函数对 x 、 y 的仿射梯度，预先求出两个 UV 差向量，与逐点重新计算这三个位置数学等价。偏移始终为一个屏幕像素，不能在高 spp 时改为一个子样本间隔，否则会低估纹理足迹。

用第 0 层宽 W 、高 H 分别缩放 UV 分量，得到 $dx=(W\Delta u_x, H\Delta v_x)$ 、 $dy=(W\Delta u_y, H\Delta v_y)$ 。足迹 $\rho=\max(\|dx\|, \|dy\|)$ ，理想层级 $D=\log_2\rho$ ；放大或常量 UV 时使用 0 层，结果夹在 $[0, \text{最大可用层}]$ 内。尺寸使用 W 、 H ，而不是某个已经缩小的层。

L_ZERO 固定采样第 0 层； $L_NEAREST$ 选择 $\text{floor}(D+0.5)$ ； L_LINEAR 分别采样 $\text{floor}(D)$ 和下一层，再按 D 的小数部分混合。与 P_LINEAR 组合时，每层内部双线性、层间再线性混合，构成三线性过滤。这属于任务 6 的完整实现。

起始 `generate_mips` 分配了最粗层却没有填充它。本次修复生成循环上界，使全部已分配层都实际过滤得到纹素；否则缩到最高 LOD 时可能错误地读到黑色。对仅一行或一列、奇数尺寸纹理也保留原框架的降采样规则。

code/src/texture.cpp

Source excerpt part 1/2 original line numbers

```
21  Color Texture::sample(const SampleParams& sp) {
22      if (mipmap.empty()) return Color(1, 0, 1);
23
24      // Pixel filtering and level filtering are independent choices.
25      const auto sample_level = [&](int level) -> Color {
26          if (sp.psm == P_NEAREST) return sample_nearest(sp.p_uv, level);
27          if (sp.psm == P_LINEAR) return sample_bilinear(sp.p_uv, level);
28          return Color(1, 0, 1);
29      };
30
31      if (sp.lsm == L_ZERO) return sample_level(0);
32
33      const float level = get_level(sp);
34      if (sp.lsm == L_NEAREST) {
35          return sample_level(static_cast<int>(std::floor(level + 0.5f)));
36      }
37      if (sp.lsm == L_LINEAR) {
38          const int lower = static_cast<int>(std::floor(level));
39          const int upper = std::min(lower + 1, static_cast<int>(mipmap.size()
> - 1));
40          const float weight = level - lower;
41          return sample_level(lower) * (1.0f - weight) + sample_level(upper) *
> weight;
42      }
43
44      // Magenta makes an invalid sampling method or texture visible.
45      return Color(1, 0, 1);
```

图 7.1 UV 足迹与层级选择的实际源码。

```
46     }

48     float Texture::get_level(const SampleParams& sp) {
49         if (mipmap.empty() || !valid_mip(mipmap[0])) return 0.0f;
50
51         // Derivatives are measured one SCREEN pixel away even with
52         > supersampling.
53         // Scale UV differences into texels of the original, full-resolution
54         > image.
55         const Vector2D dx = sp.p_dx_uv - sp.p_uv;
56         const Vector2D dy = sp.p_dy_uv - sp.p_uv;
57         const double w = static_cast<double>(mipmap[0].width);
58         const double h = static_cast<double>(mipmap[0].height);
59         const double dx_length_squared = dx.x * dx.x * w * w + dx.y * dx.y * h
60         > * h;
61         const double dy_length_squared = dy.x * dy.x * w * w + dy.y * dy.y * h
62         > * h;
63         const double footprint = std::sqrt(std::max(dx_length_squared,
64         > dy_length_squared));
65         const float max_level = static_cast<float>(mipmap.size() - 1);
66
67         // Magnification and a constant UV field use the original image.
68         if (std::isnan(footprint) || footprint <= 1.0) return 0.0f;
69         if (!std::isfinite(footprint)) return max_level;
70         return static_cast<float>(std::min(static_cast<double>(max_level),
71         > std::log2(footprint)));
72     }
```

图 7.1 续图：以第 0 层的宽高换算足迹，计算并夹取 $\log_2$ 层级。

自备 `sampling_texture.png` 为 1024×1024 图，含 $2/4/8$ 像素棋盘及 128 像素间隔双色线。`texture_demo.svg` 用 24×24 网格构造非均匀梯形，展示逐渐缩小的纹理。六图统一默认视角、 800×800 、1 spp、检查器 (400,230)。两份素材均位于 `docs/`。

7.2 六种组合对比

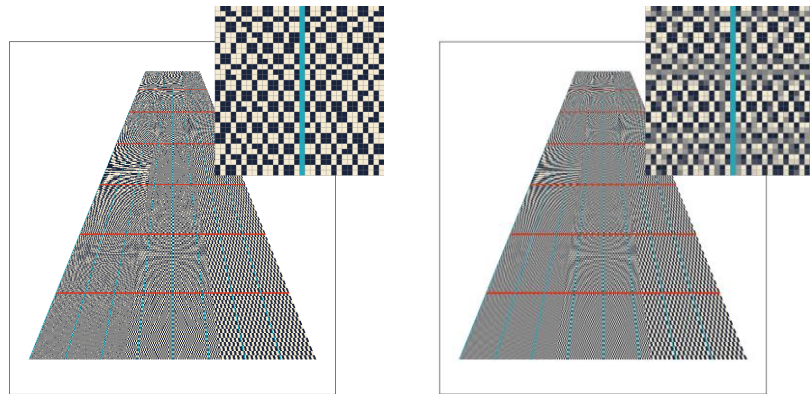


图 7.2 左 `L_ZERO+P_NEAREST`，右 `L_ZERO+P_LINEAR`；其他参数如上。

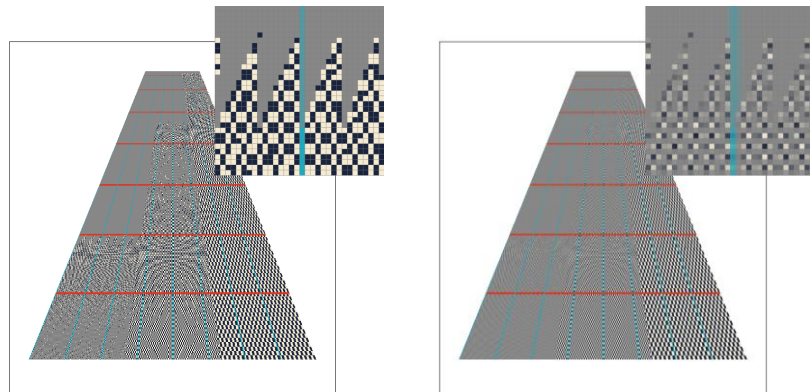


图 7.3 左 `L_NEAREST+P_NEAREST`，右 `L_NEAREST+P_LINEAR`；参数相同。

0 层细纹在缩小区形成虚假低频图案；最近层使用预过滤图像，减弱混叠，`P_LINEAR` 再平滑层内边界。四图只改变像素与层级策略。

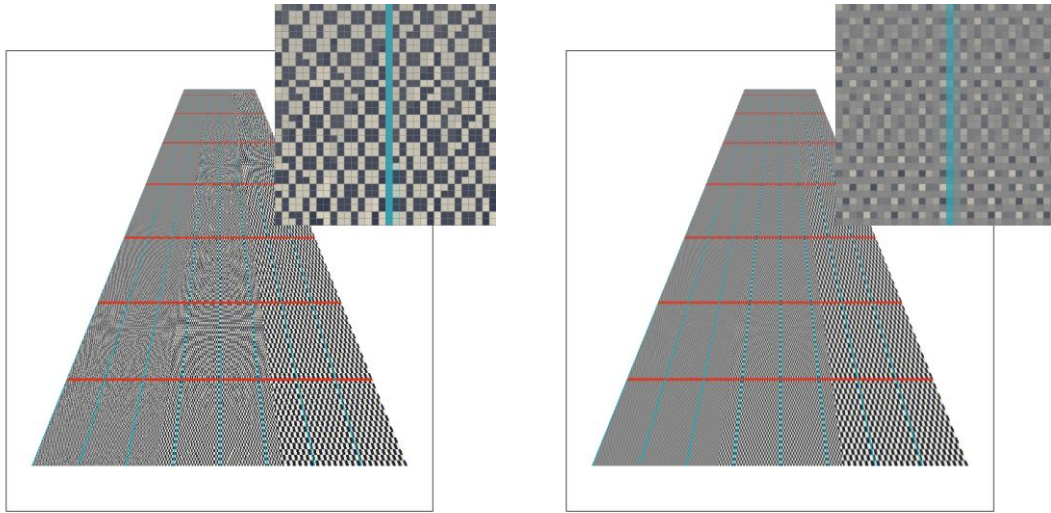


图 7.4 相同场景与参数：左 L_LINEAR+P_NEAREST，右 L_LINEAR+P_LINEAR。两张相邻层按连续 D 混合，避免整数层切换的突变。

最近邻每次读取 1 个纹素，双线性读取 4 个；最近层只使用一层，线性层级至多采样两层。以 P_LINEAR+L_LINEAR 为例，单个屏幕样本至多组合 8 个纹素，而 16 spp 还要对这一过程执行十六次。因此三种控制项可分别影响重建质量、缩小过滤和积分密度。

规则超采样缓冲随 spp 线性增长；改变 P 不增加采样缓冲，改变 L 的选择本身也不重新分配纹理。在本框架中纹理初始化即生成 MipMap，所以 L_ZERO 也保留已生成的层。对于长宽均逐级减半的常见纹理，完整金字塔像素总量约为原图的 4/3；该比例不适用于所有狭长纹理。

超采样主要改善屏幕积分，MipMap 主要抑制缩小纹理的高频混叠，双线性主要改善层内连续性。使用最近层仍可能随尺度改变出现层级跳变；三线性缓解层间变化，但各向同性足迹在斜视或高度拉伸区域可能模糊细节。各向异性过滤可作为后续改进，本次未将它列为已实现功能。

八、实践与思考

8.1 验证方法与实际修复

本次程序验证沿着覆盖判定、子样本缓冲、属性插值和纹理层级的数据流核对。首先检查顶点方向、边界、包围盒裁剪和退化三角形，再检查采样率与窗口变化时的缓冲长度；最后以小纹理的已知颜色和可计算的 UV 足迹核对像素与层级过滤，避免只凭一张图判断算法正确。

实际修复包括：统一三处缓冲分配与索引布局；让点和线写满该像素的子样本；对 8-bit 输出进行夹取和舍入；处理零长度线段、纹理边界和不合法参数；修复最后一级 MipMap 未填充。UV 梯度以一屏幕像素为单位，使改变超采样率时不会错误改变 LOD 公式。

OpenGL 实测还定位到教师 drawrend.cpp 的 glOrtho 近、远裁剪面同为 0，引发 GL_INVALID_VALUE。修复为 -1、1 后，使用窗口左上角对应的 glRasterPos2f(0,height)，移除原先的偏移，恢复有效贴屏投影。Windows 构建同时补充 WIN32 平台宏、GLEW_STATIC 及 texture.cpp 的 cstdint 引入。

算法验证记录为 verification/algorithm-tests.log，41 项全部通过、0 项失败。覆盖中心采样、顺逆时针、共享边无缝、裁剪、退化图元；4/9/16 spp 的精确半覆盖与点线广播；缓冲 resize/clear；可计算的质心权重；纹理角点、双线性均值、边界夹取；奇数与单维纹理、完整 1×1 最粗层；LOD 足迹、六组合及超采样下的一屏幕像素导数。实际截图参数、输出哈希和 OpenGL 信息留存在 verification/screenshots.json。

打包后的 bin/draw.exe 在 PATH 仅包含 Windows/System32 的环境下，对全部 32 个教师 SVG 成功生成 400×400 PNG，记录在 headless-tests.json；这也检查了运行资源与依赖。GDB 断点实际命中 rasterize_triangle 并记录调用链（gdb.log），验证 VSCode 所用调试器后端；本次未进行手动 F5 界面测试。

8.2 结果与复现

可视结果显示，单色多边形、机器人和色轮能够沿同一光栅化管线绘制；提高规则网格密度使细边缘更平滑；双线性改善纹理放大时的块感；层级采样改善

缩小高频纹理的混叠。静态 PNG 能证明本次渲染结果，不能单独证明所有交互尺度下都没有闪烁，也不能代替算法边界测试。

Windows 复现先运行项目中的构建脚本，再将 SVG 文件或目录作为 draw.exe 参数。GUI 快捷键为 -= 调整 spp、P 切换像素采样、L 切换层级采样、Z 打开像素检查器、S 保存 PNG、空格复位视角；Q/E 旋转视口，A 切换解析覆盖率，B 切换规则 SSAA 的光栅化内核。解析模式中保留的 spp 供切回 SSAA 使用，不改变解析面积计算。VSCode 配置用于选择编译与调试入口。报告所用的自动导出命令和参数另行留存在截图生成记录中。

后续可将采样缓冲按瓦片分配，降低高分辨率与高 spp 的峰值内存；也可在 SIMD 之前加入分块全内/全外测试，减少逐样本判断。解析模式可以优化多边形临时分配及全覆盖快速路径。改进时应固定场景和参数比较耗时与误差，不能把方法名称本身作为质量或性能提高的证据。

8.3 调研来源

Pineda, A Parallel Algorithm for Polygon Rasterization, SIGGRAPH 1988; 原文: <https://people.csail.mit.edu/ericchan/bib/pdf/p17-pineda.pdf>。

Hasselgren 等, Masked Software Occlusion Culling, High Performance Graphics 2016; 作者页: <https://fileadmin.cs.lth.se/graphics/research/papers/2016/culling/>。

Sean Barrett, How the stb_truetype Anti-Aliased Software Rasterizer v2 Works, 2015; 原作者说明: <https://nothings.org/gamedev/rasterize/>。

Zhou 等, Vectorization for Fast, Analytic, and Differentiable Visibility, 2021; 研究页: <https://research.nvidia.com/labs/rtr/publication/zhou2021vectorization/>。

Intel Intrinsics Guide: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>。文献用于算法和工程背景，本报告中的实验数据均来自当前源码与本机测试。